

Data Requirements and Use Cases

1. System Overview

The system is a basic e-commerce application that allows users to view products, add products to the catalog, purchase one product at a time, and review all completed orders. The application uses PHP with PDO to connect to a MySQL database named ecom.

The homepage provides navigation to three primary areas:

- Products
- Add a Product
- Orders

The system has two general actors:

- **Customer:** Browses products and purchases an item.
- **Store administrator or employee:** Adds products and reviews completed orders.

The application does not explicitly implement user roles or authentication, so the administrator or employee actor is inferred from the available product-management and order-review features.

2. Data Requirements

2.1 Product Data

The system must store information about every product offered for sale.

For each product, the system requires:

- A unique product identifier
- Product name
- Product description
- Product price
- Current quantity in stock

The Add Product page collects the name, description, price, and stock quantity from the user. All four fields are required by the form.

When a product is submitted, the system inserts the product name, description, price, and stock quantity into the product database.

Product business rules

1. Each product must have a unique product ID.
2. A product must have a name.
3. A product must have a description.
4. A product must have a price.
5. A product must have a stock quantity.
6. The product price and stock quantity must be numeric values.
7. The system must make stored products available on the Products page.
8. A product can only be purchased when its stock quantity is greater than zero.
9. Each successful purchase reduces the product's stock quantity by one.
10. The product record must remain available so that completed orders can display the product name.

The current implementation retrieves all product records and displays each product's name, description, price, and stock quantity.

2.2 Customer Data

The system must collect and store customer information when a purchase is made.

For each customer, the system requires:

- A unique customer identifier
- First name
- Last name
- Email address

The checkout form requires the customer to enter a first name, last name, and email address.

Customer business rules

1. A customer must provide a first name.

2. A customer must provide a last name.
3. A customer must provide an email address.
4. The email address is used to determine whether the customer already exists.
5. Customer email addresses must be unique.
6. When the submitted email address does not already exist, the system creates a new customer.
7. When the submitted email address already exists, the system retrieves and reuses the existing customer ID.
8. A customer can make multiple purchases.

When checkout is submitted, the system first attempts to insert a new customer. If insertion fails because the email already exists, it retrieves the customer ID associated with that email.

2.3 Credit Card Data

The system must collect and store payment-card information during checkout.

For each credit card, the system requires:

- A unique card identifier
- The customer associated with the card
- Card number
- Expiration month and year

The checkout page requires the customer to provide a card number and expiration month.

Credit card business rules

1. A card must belong to an existing customer.
2. A card number is required to complete checkout.
3. An expiration month and year are required.
4. The submitted expiration value must be converted into a date before being stored.
5. A new credit-card record is created each time a purchase is completed.
6. A customer may therefore have multiple stored credit-card records.
7. Stored card information must be available when completed orders are displayed.

The application appends the first day of the selected expiration month before storing the value as a date.

2.4 Purchase Data

The system must create a purchase record whenever a customer successfully buys a product.

For each purchase, the system requires:

- A unique purchase identifier
- The customer who made the purchase
- The purchase date
- The total purchase amount

Purchase business rules

1. Every purchase must belong to one customer.
2. Each purchase must have a unique purchase ID.
3. The purchase total must equal the price of the selected product.
4. The purchase date must be recorded by the database when the purchase is created.
5. A purchase must be created only after the customer and payment information have been processed.
6. The purchase must be associated with the product that was bought.
7. The purchase process must occur within a database transaction.
8. The transaction must be committed only after the related customer, credit-card, purchase, purchase-item, and stock operations have been attempted.

The checkout process begins a database transaction, creates the purchase, obtains its generated ID, updates the related purchase-item data, adjusts stock, and then commits the transaction.

2.5 Purchase-Item Data

The system must store the product associated with each purchase.

For each purchased item, the system requires:

- The related purchase ID
- The related product ID
- The product price at the time of purchase
- The amount purchased

Purchase-item business rules

1. Each purchase item must refer to an existing purchase.
2. Each purchase item must refer to an existing product.
3. The system must preserve the product's price at the time the purchase is made.
4. The current implementation records an amount of one for every purchase.
5. A completed purchase therefore contains one product and one unit of that product.
6. The purchase-item information must remain available for the Orders page.
7. Historical purchase prices must remain unchanged even if the current product price is later updated.

The application inserts the purchase ID, product ID, current product price, and a fixed amount of one into the purchase-item data.

2.6 Inventory Data

The system must track the number of units currently available for each product.

Inventory business rules

1. The system must retrieve a product's current stock before processing checkout.
2. A product is considered available when its stock is greater than zero.
3. The system must reject an attempt to purchase a product with zero stock.
4. A successful purchase must reduce stock by exactly one.
5. The updated stock value must be stored before the purchase transaction is committed.
6. The current stock quantity must be displayed on the Products page.

Before showing the checkout form, the application checks whether the selected product's stock is greater than zero. If it is not, the application displays an out-of-stock message and ends the request.

After the purchase is created, the application retrieves the current stock, subtracts one, and updates the product record.

2.7 Order-Display Data

The system must retrieve and display complete information about all recorded orders.

For each displayed order, the system requires:

- Purchase ID
- Purchase date
- Purchase total
- Customer ID
- Customer first name
- Customer last name
- Customer email
- Credit-card ID
- Credit-card number
- Credit-card expiration date
- Product ID
- Product name
- Price at purchase
- Amount purchased

The Orders page combines purchase, customer, credit-card, purchase-item, and product information and sorts the results by purchase ID in descending order.

Order-display business rules

1. All completed orders must be retrievable.
2. The most recently created orders must appear first.
3. Each displayed order must show its purchase date and total.
4. Each displayed order must show the customer's name and email.
5. Each displayed order must show the card number and expiration date.
6. Each displayed order must show the product name, purchase-time price, and amount.
7. When no orders exist, the system must display a "No orders on record" message.
8. Viewing orders does not modify database records.

The application constructs an order display containing the purchase, customer, card, and product details for every returned record.

3. Data Integrity Requirements

The system should enforce the following requirements based on how the PHP code uses the data:

1. Every product must have a valid product ID.
2. Every customer must have a valid customer ID.
3. Every purchase must have a valid purchase ID.
4. Every credit-card record must reference an existing customer.
5. Every purchase must reference an existing customer.
6. Every purchase item must reference an existing purchase and product.
7. Customer email addresses must be unique.
8. Required checkout fields must not be empty.
9. Required product fields must not be empty.
10. A purchase cannot be completed for a product that does not exist.
11. A purchase cannot be completed for a product whose stock is zero.
12. A successful purchase must reduce stock by one.
13. The purchase price must be saved separately from the product's current price.
14. Customer, payment, purchase, purchase-item, and stock changes should be treated as one transaction.
15. If the purchase process cannot be completed, partial purchase data should not remain in the database.

The forms use HTML required attributes for customer, payment, and product-entry fields.

4. Actors

4.1 Customer

The customer interacts with the purchasing portion of the application.

The customer can:

- View all products
- Review product descriptions, prices, and stock
- Select a product to buy
- Enter personal information
- Enter payment-card information
- Complete a purchase
- Receive confirmation that a purchase succeeded

4.2 Store Administrator or Employee

This actor is inferred from the system's product-entry and order-review pages. The code does not enforce a separate administrator account or role.

The administrator or employee can:

- Add a new product
 - View the current product catalog
 - View all completed orders
 - Review customer, payment, product, and purchase information
-

5. Customer Use Cases

UC-01: View Products

Primary actor: Customer

Description:

The customer views all products currently stored in the e-commerce system.

Preconditions:

- The system is connected to the database.
- Product records may or may not exist.

Trigger:

- The customer opens the Products page.

Main flow:

1. The customer selects the Products navigation link.
2. The system retrieves all records from the products data.
3. The system processes each returned product.
4. The system displays the product name.
5. The system displays the product description.
6. The system displays the product price.
7. The system displays the current stock quantity.
8. The system provides a Buy Item button for the product.

The Products page retrieves all products and dynamically creates the display and purchase form for each product.

Postconditions:

- The product catalog is displayed.
 - No database data is changed.
-

UC-02: Select Product for Purchase

Primary actor: Customer

Description: The customer selects one product from the product catalog to purchase.

Preconditions:

- The selected product appears on the Products page.

Trigger:

- The customer clicks the Buy Item button.

Main flow:

1. The customer reviews the available products.
2. The customer clicks Buy Item for one product.
3. The system submits the selected product ID to the checkout page.
4. The system retrieves the selected product from the database.
5. The system verifies that the product exists.
6. The system checks whether the product has stock available.
7. The system displays the selected product's name and price.
8. The system displays the checkout form.

The Buy Item form submits the selected product_id to the checkout page.

Alternative flow A — Product does not exist:

1. The system cannot find a product matching the submitted ID.
2. The system displays a message stating that the selected item does not exist.
3. The system provides a link back to the Products page.
4. The checkout process ends.

Alternative flow B — Product is out of stock:

1. The system finds the selected product.
2. The system determines that its stock is not greater than zero.
3. The system displays an out-of-stock message.
4. The system provides a link back to the Products page.
5. The checkout process ends.

Alternative flow C — No product was selected:

1. The checkout page is accessed without a valid product-selection request.
2. The system displays a message stating that an item must be selected.
3. The system provides a link back to the Products page.
4. The checkout process ends.

These validation paths are implemented before the checkout form is displayed.

Postconditions:

- The selected product is ready for checkout.
 - No purchase has yet been recorded.
-

UC-03: Enter Checkout Information

Primary actor: Customer

Description: The customer enters the personal and payment information needed to purchase the selected item.

Preconditions:

- A valid product has been selected.

- The selected product is in stock.

Trigger:

- The system displays the checkout page.

Main flow:

1. The system shows the selected product's name and price.
2. The customer enters a first name.
3. The customer enters a last name.
4. The customer enters an email address.
5. The customer enters a card number.
6. The customer selects the card expiration month and year.
7. The system includes the selected product ID as hidden form data.
8. The system includes the selected product price as hidden form data.
9. The customer selects Make Purchase.

The checkout form includes all five required customer and payment fields, along with hidden product ID and product-price fields.

Alternative flow — Missing information:

1. The customer leaves a required field empty.
2. The browser prevents the form from being submitted.
3. The customer must enter the missing information before continuing.

Postconditions:

- The checkout data is submitted for purchase processing.
-

UC-04: Identify or Create Customer

Primary actor: Customer

Supporting system: Database

Description: The system identifies an existing customer by email or creates a new customer during checkout.

Preconditions:

- The customer has submitted the checkout form.
- A first name, last name, and email address were provided.

Trigger:

- The customer selects Make Purchase.

Main flow:

1. The system attempts to insert the submitted first name, last name, and email address.
2. The database accepts the insertion.
3. The system retrieves the newly generated customer ID.
4. The new customer ID is used for the rest of the purchase process.

Alternative flow — Existing customer:

1. The customer's email address already exists.
2. The attempted customer insertion does not succeed.
3. The system searches for a customer with the submitted email.
4. The system retrieves the existing customer ID.
5. The existing customer ID is used for the rest of the purchase process.

The code uses the email address to prevent duplicate customer records and retrieve an existing customer when necessary.

Postconditions:

- A valid customer ID is available.
 - The customer is either newly created or matched to an existing record.
-

UC-05: Record Payment Card

Primary actor: Customer

Description:

The system stores the card information supplied by the customer during checkout.

Preconditions:

- A valid customer ID is available.

- The customer has entered a card number and expiration month.

Trigger:

- Purchase processing continues after the customer is identified.

Main flow:

1. The system receives the submitted card number.
2. The system receives the submitted expiration month and year.
3. The system converts the expiration value into a date.
4. The system associates the card with the customer ID.
5. The system inserts the card information into the database.

Postconditions:

- A credit-card record is stored for the customer.
- The purchase process continues.

The implementation stores the submitted card information after resolving the customer ID.

UC-06: Complete Product Purchase

Primary actor: Customer

Description: The customer completes the purchase of one selected product.

Preconditions:

- A valid product was selected.
- The product is in stock.
- The customer submitted all required checkout information.
- A valid customer ID is available.

Trigger:

- The customer selects Make Purchase.

Main flow:

1. The system begins a database transaction.

2. The system identifies or creates the customer.
3. The system stores the submitted credit-card information.
4. The system creates a purchase associated with the customer.
5. The system records the selected product's price as the purchase total.
6. The system retrieves the newly generated purchase ID.
7. The system creates a purchase-item record.
8. The system associates the purchase item with the selected product.
9. The system records the product price at the time of purchase.
10. The system records the amount purchased as one.
11. The system retrieves the product's current stock quantity.
12. The system subtracts one from the stock quantity.
13. The system updates the product's stock.
14. The system commits the transaction.
15. The system displays a purchase-successful message.
16. The system provides a link back to the Products page.

The application performs the customer, card, purchase, purchase-item, and stock operations inside one transaction.

Postconditions:

- A purchase record exists.
 - A purchase-item record exists.
 - The product stock has decreased by one.
 - The customer's card information has been stored.
 - The user receives a successful-purchase confirmation.
-

6. Administrator or Employee Use Cases

UC-07: Add Product

Primary actor: Store administrator or employee

Description:

The actor adds a new product to the store's product catalog.

Preconditions:

- The application can connect to the database.

- The actor has opened the Add a Product page.

Trigger:

- The actor selects the Add a Product navigation link.

Main flow:

1. The system displays the Add a Product form.
2. The actor enters the product name.
3. The actor enters the product description.
4. The actor enters the product price.
5. The actor enters the starting stock quantity.
6. The actor selects Add Product.
7. The system retrieves the submitted form values.
8. The system prepares an insert operation.
9. The system inserts the product into the database.
10. The product becomes available through the Products page.

The product form requires all four values, and the PHP code inserts them into the product data.

Alternative flow — Missing value:

1. The actor leaves a required field empty.
2. The browser prevents submission.
3. The actor must complete the missing field.

Postconditions:

- A new product is stored.
- The product can be displayed in the product catalog.

UC-08: View All Orders

Primary actor: Store administrator or employee

Description: The actor views all purchases recorded in the system.

Preconditions:

- The system can connect to the database.
- At least one order may exist.

Trigger:

- The actor selects the Orders navigation link.

Main flow:

1. The system retrieves all purchase records.
2. The system joins each purchase with its customer information.
3. The system joins each customer with stored card information.
4. The system joins each purchase with its purchase-item information.
5. The system joins each purchase item with its product information.
6. The system sorts the results by purchase ID in descending order.
7. The system displays the order ID.
8. The system displays the purchase date and total.
9. The system displays the customer's name and email.
10. The system displays the card number and expiration date.
11. The system displays the product name.
12. The system displays the price at purchase.
13. The system displays the amount purchased.
14. The process repeats for every returned order.

The Orders page retrieves information from all five major data areas and displays the combined order details.

Alternative flow — No orders exist:

1. The query returns no orders.
2. The system displays "No orders on record."
3. The system provides a link back to the homepage.
4. The page stops further processing.

Postconditions:

- All available orders are displayed.
- No database data is changed.

UC-09: Navigate Between System Pages

Primary actor: Customer, administrator, or employee

Description: The actor moves between the major areas of the application.

Preconditions:

- The actor has opened the homepage, Products page, Add Product page, or Orders page.

Main flow:

1. The system displays the navigation menu.
2. The actor selects Products to open the catalog.
3. The actor selects Add a Product to open the product-entry page.
4. The actor selects Orders to open the completed-order page.

The same three navigation options appear throughout the main system pages.

Postconditions:

- The selected page is displayed.
- No database information is changed solely from navigation.